



MCU-Friendly Mamba
Optimizing Mamba Model Inference on Microcontrollers

Bartosz Drabiński¹

Supervisor: Bo Yang¹
Responsible Professor: Qing Wang¹

¹EEMCS, Delft University of Technology, The Netherlands

A Thesis Submitted to EEMCS Faculty Delft University of Technology,
In Partial Fulfilment of the Requirements
For the Bachelor of Computer Science and Engineering
May 22, 2026

Name of the student: Bartosz Drabiński
Final project course: CSE3000 Research Project
Thesis committee: dr. Qing Wang, Mark Neerincx

An electronic version of this thesis is available at <http://repository.tudelft.nl/>.

Abstract

Machine learning is becoming more and more widespread. While the frontier models grow in size, and their compute requirements, for Edge AI the amount of processing power and memory are usually strictly limited. This is especially true for TinyML, a subset that specializes in running machine learning on microcontrollers (MCUs).

This trend requires new architectures, as well as adaptations of existing ones, so that they can operate within strict memory and computational limits. The Mamba architecture, which is created around State-Space Model (SSM), is particularly promising in this setting because of its compact parameterization and strong performance on long-context tasks. This work surveys recent progress in deploying Mamba models on microcontrollers and proposes architecture and optimization strategies designed specifically for MCU constraints.

Keywords: TinyML, Mamba, Microcontroller, Edge AI

1 Introduction

Recent years have shown that machine learning has immense potential for solving tasks that have been challenging to accomplish using traditional algorithms. This led to creation of many general models, such as Large Language Models, as well as specialized ones for one task.

This has led to wide spread adoption in use of those models. While many of them have adopted a centralized approach which utilized large data centres efficiently processing queries received from users all across the world, this approach also has its limitations. Some of the main ones, are the need for the device to communicate over internet, which also adds latency, as well as privacy concerns.

Edge AI tries to improve those limitations, by miniaturizing the models, which allows them to be run on consumer devices. A subset of it, TinyML, focuses on the smallest networks, which can be deployed on microcontrollers. This introduces various limitations. MCUs lack the processing power required to run large models, and even memory to be able to save their weights. A microcontroller usually provides us a single, or dual-core CPU with a frequency below 240 MHz, and 512 KB RAM. The latency of the model is also important, as many sources of data for this application come in streams, like audio, or video data, so it would preferably be able to process it in real time.

There have been multiple machine learning architectures created over the years such as Multi Layer Perceptrons (MLP), Convolutional Neural Networks (CNN), Recurrent Neural Network (RNN), or Transformers. Recently a new one has been proposed - Mamba architecture [?] - and has emerged as a compelling alternative to the dominant Transformer models. It works by combining advantages of CNNs and RNNs, which come with many different advantages, including a smaller memory footprint and robust performance

on long-context tasks. It achieves that by utilizing a Selective State-Space Model (S4) block.

Optimizing Mamba architecture has been explored with Quantization [], Pruning, and Knowledge Distillation [] for normal deployment. Implementing and running inference on MCUs has also been achieved in MambaLite-Micro [] paper. The authors of it replicated directly the pytorch inference code for mamba model in pure C code and run it on ESP32S4 and STM32 microcontrollers. This approach makes it possible to achieve full compability with the pytorch version, but also comes with many drawbacks. Especially with difficulties for introducing changes in architecture, or introducing optimizations like the ones mentioned previously.

This paper tries to address this gap in research by answering a question: How can Mamba be improved for microcontroller inference. This is done by investigating possible ways to deploy this model, looking for bottlenecks both in memory usage, as well as latency. Additionally we check if the optimizations introduced for GPU inference have positive impact on MCU inference.

The ?? describes in more detail previous works done on this topic. Then in ?? we explain how the experimental setup was chosen. Next in ?? we describe the deployment pipeline of our mamba model. In ?? we describe the results of the conducted experiments. Next in ?? we go over the ethical implications of our work. After this in ?? we interpret the results. Finally in ?? we conclude the research and give future work recommendations.

2 Related Work and Background

2.1 TinyML

2.2 Mamba architecture

, which allows for effective capturing of long-range dependencies, while it reduces quadratic complexity of attention mechanisms to linear scaling. SSM works by splitting the context and processing it while preserving a hidden state:

$$\begin{aligned}h_t &= \overline{A}h_{t-1} + \overline{B}x_t \\y_t &= C\overline{h}_t + D\overline{x}_t\end{aligned}$$

This operation then can be performed efficiently on a modern GPU by performing a parallel scan operation. Mamba however poses a few challenges, primarily due to difficulties in quantization [?; ?] and a lack of standardized tooling, e.g. easy export to ONNX [?].

The advantages, however, make Mamba a promising candidate for TinyML applications, a domain experiencing rapid growth. Transitioning Mamba from GPU-centric environments to resource-constrained MCUs requires addressing critical gaps. Recent literature highlights two primary hurdles: model quantization and extreme memory minimization. Quantizing SSMs is notoriously difficult due to their sensitivity to outliers, often necessitating advanced Post-Training Quantization (PTQ) [?] or Quantization-Aware Training (QAT) [?]. Furthermore, achieving the footprint reduction required for MCU deployment remains a significant challenge [?], with some efforts focusing on bridging the lack of native ONNX support to enable deployment on physical hardware [?].

2.3 Research Gap

This section should provide the theoretical and research background necessary to understand the paper.

This section should support the motivation for the experimental framework introduced later in the paper.

3 Methodology

This section should describe the experimental framework and explain how the research question is investigated.

3.1 Dataset and Preprocessing

Describe:

- Dataset selection

3.2 Experimental Design

Describe:

- Quantization strategies
- Memory optimization strategies
- Ways of measuring used memory
- Testing strategies and reliability
- Repeated runs and statistical stability

3.3 Model Architecture

Describe:

- Changes made to the original
- Hyperparameter choices

3.4 Deployment Optimization

Describe:

- Deployment choices
- INT8, INT4 quantization
- Quantization-aware training
- Latency benchmarking

4 Embedded Deployment Setup

This section should describe the embedded-oriented deployment environment and evaluation setup.

4.1 Software Stack

Describe:

- No platform IO
- No ONNX
- C library
- Pytorch
- Brevitas
- Torchao
- Other relevant libraries

4.2 Deployment Measurements

Explain:

- Model size measurement
- Memory throughout runtime measurement
- Latency benchmarking methodology
- Inference execution setup

4.3 Reproducibility

Describe:

- Random seeds
- Repeated runs
- Configuration consistency
- Code availability

5 Results

This section presents the experimental results of the impact of different techniques on the accuracy and memory usage. Results should be supported by descriptive and clear figures.

5.1 Effect of Quantization on Memory and Accuracy

This subsection presents:

- Memory reductions across quantization strategies
- Accuracy changes induced by quantization
- Variance of performance across repeated runs

5.2 Embedded Efficiency Trade-offs

This subsection presents:

- Inference latency measurements
- Model size comparisons
- Accuracy-efficiency relationships

5.3 Effects of other memory optimizations on accuracy and latency

This subsection presents:

- Impacts of Low rank factorization

6 Responsible Research

This section discusses the ethical implications of the work, reproducibility considerations, and the role of AI-assisted tools in the development process.

6.1 Ethical Considerations

Key ethical aspects include:

- Possibility of reducing power usage
- Enabling bad actors to run dangerous ML models on embedded devices (listening)

6.2 Reproducibility

The following aspects support reproducibility:

- Publicly available dataset and deterministic preprocessing pipeline
- Ease of reproducing the model training environment
- Access to the microcontroller version used
- Consistent evaluation metrics across all experimental runs
- Code structured to enable rerunning of full experimental pipelines

6.3 Use of AI Tools

Artificial intelligence tools were used as supportive instruments during the development of this work. Their use was limited to improving code readability, assisting with debugging, and refining written text for clarity and grammatical correctness.

Specifically, AI assistance was used for:

- Debugging implementation issues in experimental code
- Improving clarity and structure of written explanations
- Assisting with formatting and consistency in LaTeX writing

All scientific decisions, experimental design choices, and interpretation of results were made independently by the author. AI tools were not used to generate or alter experimental outcomes, results, or conclusions.

7 Discussion

This section interprets the experimental findings and discusses their implications for embedded trustworthy AI systems. It also discusses the limitations of the approach.

7.1 Trade offs between accuracy, memory efficiency, and latency

Discuss:

- How making models smaller reduces accuracy of them
- How some optimizations can be made that hurt latency

7.2 How Mamba model performs

Discuss:

- The performance of Mamba compared to SOTA for the dataset

7.3 Limitations

Discuss:

- Dataset limitations
- More quantization possible
- Hardware specific implementation
- Difficulty of comparison with other models
- Lack of standardized tooling

8 Conclusions and Future Work

This section should directly answer the research question and summarize the main findings.

The conclusion should:

- Restate the research objective
- Summarize the most important findings
- Explain the memory limitations of Mamba models on MCUs
- Highlight key trade-offs identified in the experiments

Future work may discuss:

- Other memory optimization strategies
- More standardized deployment (LiteRT custom operation)
- Energy-aware optimization

The conclusion should remain concise and readable as a standalone section.

A Some further guidelines that go without saying (right?)

- Follow the required page limit and formatting constraints (maximum 8 pages, two-column format).
- Ensure that the paper clearly communicates the central research goal and contributions.

A.1 Reference use

- Use a numbered citation style (IEEE format), where references appear as [?], [2]...
- Use BibTeX or a similar tool to automatically generate bibliographic information.
- Every citation in the text must appear in the bibliography, and no uncited references should be included.
- All ideas, figures, and data derived from other work must be properly referenced.
- Literal quotations must include quotation marks and page numbers where applicable.
- Paraphrased content must be sufficiently distinct from the original source.
- Plagiarism is strictly prohibited.

A.2 Structure

Paragraphs

- are well-constructed and logically organized
- are not too long: each paragraph discusses a single idea
- start with clear topic sentences
- contribute to a coherent flow from research question to conclusions
- support claims with appropriate references and evidence

A.3 Style

- Use a formal academic tone throughout the paper
- Prefer “we” over “I” when referring to the authors
- Avoid informal language and subjective phrasing
- Avoid unsupported or overly strong claims
- Be precise and cautious in wording (e.g., do not claim “privacy achieved” when only proxies are used)
- Ensure correct use of English: clear grammar, no spelling or lexical errors
- Maintain clarity: avoid overly complex or ambiguous sentences
- Use varied sentence structure with a mix of active and passive voice

A.4 Tables and figures

- All tables and figures must be numbered (e.g., Figure 1, Table 1)
- Each must include a clear and descriptive caption
- Every table and figure must be referenced at least once in the text
- Figures and tables must be interpretable independently of the main text
- Avoid overly dense or cluttered visualizations
- If adapted or copied, include proper references

A.5 Page limit and formatting

- The paper must not exceed 8 pages in two-column format
- Use consistent font, spacing, and layout throughout the document
- Follow a standard conference-style formatting
- Prioritize clarity and conciseness over excessive detail

A.6 Key focus of the paper

- The paper must clearly demonstrate how embedded deployment constraints (such as model size and latency) reshape memory–accuracy–latency trade-offs in AI systems
- All sections should support and contribute to this central theme